

Kernel Normalized Convolutional Networks

Reza Nasirigerdeh

Technical University of Munich, Germany

Reihaneh Torkzadehmahani

Technical University of Munich, Germany

Daniel Rueckert *

Technical University of Munich, Germany
Imperial College London, United Kingdom

Georgios Kaissis *

Technical University of Munich, Germany
Helmholtz Zentrum Munich, Germany

Abstract

Existing deep convolutional neural network (CNN) architectures frequently rely upon batch normalization (BatchNorm) to effectively train the model. BatchNorm significantly improves model performance in centralized training, but it is unsuitable for federated learning and differential privacy settings. Even in centralized learning, BatchNorm performs poorly with smaller batch sizes. To address these limitations, we propose *kernel normalization* and *kernel normalized convolutional* layers, and incorporate them into *kernel normalized convolutional networks* (KN-ConvNets) as the main building blocks. We implement KNConvNets corresponding to the state-of-the-art CNNs such as VGGNets and ResNets while forgoing BatchNorm layers. Through extensive experiments, we illustrate KNConvNets consistently outperform their batch, group, and layer normalized counterparts in terms of both accuracy and convergence rate in centralized, federated, and differentially private learning settings.

1 Introduction

Convolutional neural networks (CNNs) [1] are standard architectures in computer vision tasks such as image classification [2, 3] and object detection [4]. Deep CNNs including ResNets [5] achieved outstanding performance in classification of challenging datasets such as ImageNet [6]. One of the main building blocks of these CNNs is *batch normalization* (BatchNorm) [7]. The BatchNorm layer considerably enhances the performance of deep CNNs by smoothening the optimization landscape [8], and addressing the problem of vanishing gradients [9, 10].

BatchNorm, however, has the disadvantage of breaking the independence among the samples in the batch [11]. This is because BatchNorm carries out normalization along the batch dimension (Figure 1a), and as a result, the normalized value associated with a given sample depends on the statistics of the other samples in the batch. In the *centralized setting* (training a single model on a centralized dataset), consequently, the effectiveness of BatchNorm is highly dependent on the batch size. Using large batch sizes, BatchNorm effectively trains the model due to more accurate estimation of the batch statistics, but incurs high memory overhead. With small batch sizes, on the other hand, BatchNorm causes a dramatic reduction in model accuracy [12].

BatchNorm, moreover, is unsuitable for *federated learning* (FL) [13], where multiple clients (e.g. hospitals or mobile devices) collaboratively train a global model under the coordination of a central server without sharing their private data [14]. This is because BatchNorm relies on the assumption of independent and identically distributed (IID) feature values in the batch [7], which is not the case for FL in which data is NonIID across the clients. BatchNorm is also inapplicable to *differential privacy* (DP) [15], where random noise is injected into the model parameters to limit the information

*Joint senior author

learnt about a particular sample in the dataset. This is due to the fact that DP requires to compute per-sample gradients [16] that is not possible for batch normalized CNNs.

To overcome the limitations of BatchNorm, *batch-independent* normalization layers including layer normalization (LayerNorm) [17], instance normalization (InstanceNorm) [18], group normalization (GroupNorm) [12], positional normalization (PositionalNorm) [19], and local context normalization (LocalContextNorm) [20] have been introduced. They perform normalization independently for each sample in the batch. While these layers do not suffer from the drawbacks of BatchNorm, they typically do not achieve performance comparable with BatchNorm’s in centralized training. In FL and DP settings, furthermore, these batch-independent normalization layers might not be as efficient as expected. This motivates us to develop alternative layers, which are batch-independent but efficient in centralized, federated, and differentially private learning environments.

Our main contribution is to propose two novel *batch-independent* layers called **kernel normalization (KernelNorm)** and **kernel normalized convolutional (KNConv)** layer to further enhance the performance of deep CNNs. KernelNorm is similar to a pooling layer, except that it normalizes the elements specified by the kernel window instead of computing the average/maximum of the elements (Figure 1g). KNConv is the combination of KernelNorm with a convolutional layer, where it applies KernelNorm to the input, and feeds KernelNorm’s output to the convolutional layer (Figure 2). From another perspective, KNConv is the same as the convolutional layer except that KNConv first normalizes the input elements specified by the kernel window, and then computes the convolution between the normalized elements and kernel weights. KNConv is computationally inefficient in both aforementioned forms because it leads to extremely large number of normalization units, and thus, considerable computational overhead to normalize the corresponding elements.

To tackle this issue, we present **computationally-efficient KNConv** (Algorithm 1), where the output of the convolution is adjusted using the mean and variance of the normalization units. This way, it is not required to normalize the elements, improving the computation time by orders of magnitude. Given KernelNorm and computationally-efficient KNConv as the main building blocks, we implement the **kernel normalized convolutional networks (KNConvNets)** corresponding to VGGNets [21] and ResNets while foregoing BatchNorm (Section 3). Using different datasets including CIFAR-10/100 [22] and Imagenette [23], we empirically demonstrate KNConvNets consistently outperform the BatchNorm, GroupNorm, and LayerNorm counterparts in terms of accuracy and convergence rate in centralized, federated, and differentially private learning settings (Section 4).

2 Normalization Layers

We provide an overview on existing normalization layers, which are based on standard normalization (i.e. zero-mean and unit-variance). For the sake of simplicity, we focus on 2D images, but the concepts are also applicable to 3D images. For a 2D image, the input of a layer is a 4D tensor of shape (n, c, h, w) , where n is the batch size, c is the number of input channels, h is height, and w is width of the tensor. Similarly, (N, C, H, W) indicate the batch, channel, height, and width dimensions, respectively. The normalization layers differ from one another in the dimensions along which they normalize the elements, or more precisely, the portion of the input tensor they consider as their *normalization unit*. The normalization unit is a group of input elements, which are normalized together with mean and variance of the unit. In the following, we review the layers from this aspect:

BatchNorm [7] (Figure 1a) normalizes the elements along dimensions (N, H, W) and independently of dimension C . BatchNorm’s normalization unit is a 3D tensor of shape (n, h, w) , i.e. BatchNorm incorporates all elements in the batch, height, and width dimensions during normalization. BatchNorm has total of c normalization units.

LayerNorm [17] (Figure 1b) carries out normalization across dimensions (C, H, W) and independently of dimension N . The normalization unit of LayerNorm is a 3D tensor of shape (c, h, w) , implying that it considers all elements in the channel, height, and width dimensions for normalization. The total number of normalization units in LayerNorm is n .

InstanceNorm [18] (Figure 1c) normalizes the elements along dimensions (H, W) and independently of dimensions (N, C) . The normalization unit of InstanceNorm is a 2D tensor of shape (h, w) , i.e. all elements of the height and width dimensions are taken into account during normalization. InstanceNorm has $n \cdot c$ normalization units.

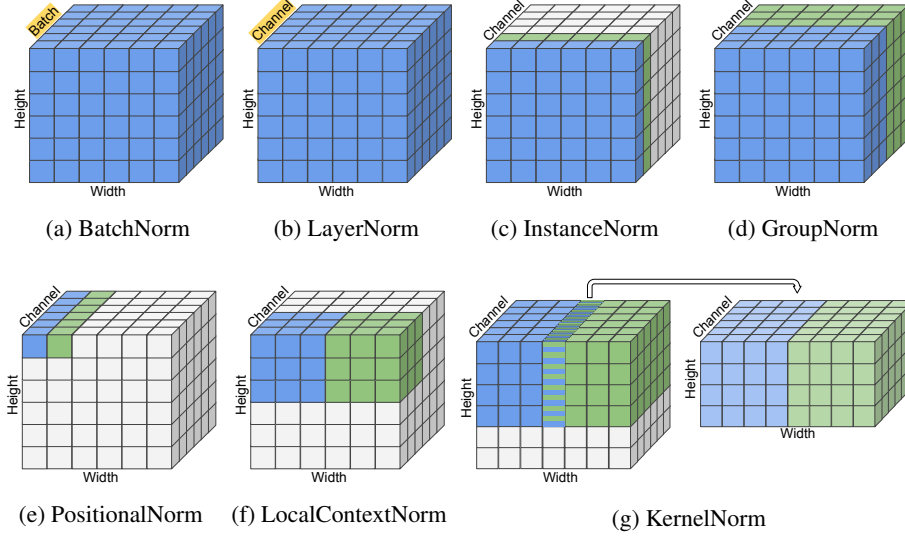


Figure 1: **Normalization layers** differ from one another in their normalization unit (highlighted in blue and green). The normalization layers in (a)-(f) establish a *one-to-one correspondence* between the input and normalized elements (i.e. no overlap between the normalization units, and no ignorance of an element). The proposed **KernelNorm** layer does not impose such one-to-one correspondence: Some elements (dash-hatched area) are common among the normalization units, contributing more than once to the output, while some elements (uncolored ones) are ignored during normalization. KernelNorm’s output shape can be different from the input shape, unlike other normalization layers. It implies KernelNorm might downsample or oversample the input tensor in addition to normalization.

GroupNorm [12] (Figure 1d) performs normalization across dimensions (C, H, W) and independently of dimension N similar to LayerNorm. However, its normalization unit is a 3D tensor of shape (c_g, h, w) , where c_g indicates the channel group size. Thus, all elements in the height and width dimensions and a subset of elements specified by the group size in the channel dimension are incorporated during normalization. The number of normalization units in GroupNorm is $n \cdot \frac{c}{c_g}$.

PositionalNorm [19] (Figure 1e) normalizes the elements along dimension C and independently of dimensions (N, H, W) . Therefore, its normalization unit is a 1D tensor of shape c . PositionalNorm has $n \cdot h \cdot w$ normalization units.

LocalContextNorm [20] (Figure 1f) conducts normalization across dimensions (C, H, W) similar to GroupNorm. However, it considers a *subset of elements* in the height, width, and channel dimensions as the normalization unit. So, LocalContextNorm’s normalization unit is a 3D tensor of shape (c_g, r, s) , where c_g is the group size, and (r, s) is the window size. It has $\frac{c}{c_g} \cdot \lceil \frac{h}{r} \rceil \cdot \lceil \frac{w}{s} \rceil$ normalization units.

BatchNorm, LayerNorm, InstanceNorm, and GroupNorm consider *all elements* in the height and width dimensions for normalization, and thus, they are referred to as *global normalization layers*. PositionalNorm and LocalContextNorm, on the other hand, are called *local normalization layers* [20] because they incorporate a *subset of elements* from the aforementioned dimensions during normalization. In spite of their differences, the aforementioned normalization layers including BatchNorm have at least two things in common: (I) There is a *one-to-one correspondence* between the original elements in the input and the normalized elements in the output. That is, there is exactly one normalized element associated with each input element. Therefore, these layers do not modify the shape of the input during normalization. (II) They require per-channel shift and scale trainable parameters to ensure that the layer output still represent identity transformation [7].

3 Kernel Normalized Convolutional Networks

KNConvNets consist of KernelNorm and KNConv layers as their main building blocks. In the following, we present these layers and KNConvNets in more detail.

KernelNorm takes the kernel size (k_h, k_w) , stride (s_h, s_w) , padding (p_h, p_w) , and dropout probability p as hyper-parameters. It pads the input with zeros if padding is specified. KernelNorm carries out normalization along the channel, height, and width dimensions but independently of the batch dimension (Figure 1g). The normalization unit of KernelNorm is a tensor of shape (c, k_h, k_w) , i.e. *all elements* in the channel dimension but a *subset of elements* specified by the kernel window from the height and width dimensions. KernelNorm (1) applies random dropout [24] to the normalization unit to obtain the *dropped-out* unit, (2) computes mean and variance of the dropped-out unit, and (3) employs the mean and variance to normalize the *original* normalization unit:

$$U' = D_p(U), \quad (1)$$

$$\mu_{u'} = \frac{1}{c \cdot k_h \cdot k_w} \cdot \sum_{i_c=1}^c \sum_{i_h=1}^{k_h} \sum_{i_w=1}^{k_w} U'(i_c, i_h, i_w), \quad (2)$$

$$\sigma_{u'}^2 = \frac{1}{c \cdot k_h \cdot k_w} \cdot \sum_{i_c=1}^c \sum_{i_h=1}^{k_h} \sum_{i_w=1}^{k_w} (U'(i_c, i_h, i_w) - \mu_{u'})^2, \\ \hat{U} = \frac{U - \mu_{u'}}{\sqrt{\sigma_{u'}^2 + \epsilon}}, \quad (3)$$

where p is the dropout probability, D_p is the dropout operation, U is the normalization unit, U' is the dropped-out unit, $\mu_{u'}$ and $\sigma_{u'}^2$ are the mean and variance of the dropped-out unit, respectively, and $\hat{U}$ is the normalized unit.

Partially inspired by BatchNorm, KernelNorm introduces a *regularizing effect* during training by intentionally normalizing the elements of the original unit U using the statistics computed over the dropped-out unit U' . In BatchNorm, the normalization statistics are computed over the batch but not the whole dataset, where the mean and variance of the batch are randomized approximations of those from the whole dataset. The “stochasticity from the batch statistics” creates a regularizing effect in BatchNorm according to [17]. KernelNorm employs dropout to generate a similar stochasticity in the mean and variance of the normalization unit. Notice that the naive option of injecting random noise directly into the mean and variance might generate too much randomness, and consequently, hinder model convergence. Using dropout in the aforementioned fashion, KernelNorm can control the regularization effect with more flexibility.

The first normalization unit of KernelNorm is bounded to a window specified by diagonal points $(1, 1)$ and (k_h, k_w) in the height and width dimensions. The coordinates of the next normalization unit are $(1, 1 + s_w)$ and $(k_h, k_w + s_w)$, which are obtained by sliding the window s_w elements along the width dimension. If there are not enough elements for kernel in the width dimension, the window is slid by s_h elements in the height dimension, and the above procedure is repeated. Notice that KernelNorm works on the padded input of shape $(n, c, h + 2 \cdot p_h, w + 2 \cdot p_w)$, where (p_h, p_w) is the padding size. The output $\hat{X}$ of KernelNorm is the concatenation of the *normalized units* $\hat{U}$ from Equation 3 along the height and width dimensions. As one might notice, KernelNorm behaves similarly to the pooling layers with a major difference: KernelNorm normalizes the elements specified by the kernel window instead of computing the maximum/average over the elements. KernelNorm’s output is of shape (n, c, h_{out}, w_{out}) , and it has total of $n \cdot \frac{h_{out}}{k_h} \cdot \frac{w_{out}}{k_w}$ normalization units, where h_{out} and w_{out} are computed as follows:

$$h_{out} = k_h \cdot \lfloor \frac{h + 2 \cdot p_h - k_h}{s_h} + 1 \rfloor, w_{out} = k_w \cdot \lfloor \frac{w + 2 \cdot p_w - k_w}{s_w} + 1 \rfloor$$

KernelNorm is a *batch-independent* and *local normalization* layer, but differs from the existing normalization layers in two aspects: (I) There is not necessarily a one-to-one correspondence between the original elements in the input and the normalized elements in the output of KernelNorm. If stride is less than kernel size, some input elements contribute more than once in the output. For stride values greater than kernel sizes, some input elements are completely ignored during normalization. Therefore, the output shape of KernelNorm can be very different from the input shape. (II) KernelNorm does not require shift and scale trainable parameters because its output is typically given to a convolutional layer but not non-linear activation functions such as ReLU.

KNConv is the combination of KernelNorm and the traditional convolutional layer (Figure 2). It takes the number of input channels ch_{in} , number of output channels (filters) ch_{out} , kernel size (k_h, k_w) , stride (s_h, s_w) , and padding (p_h, p_w) , exactly the same as the convolutional layer, as well as

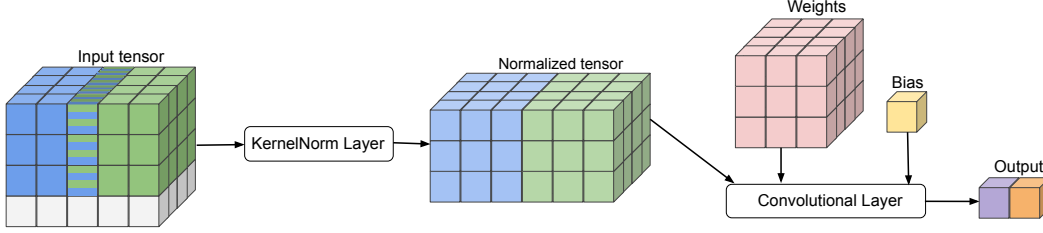


Figure 2: **KNConv** as the combination of the KernelNorm and convolutional layers. KNConv first applies KernelNorm with kernel size (3, 3) and stride (2,2) to the input tensor, and then gives KernelNorm’s output to a convolutional layer with kernel size and stride (3, 3). That is, the kernel size and stride of the convolutional layer and the kernel size of KernelNorm are identical.

the dropout probability p as hyper-parameters. KNConv first applies KernelNorm with kernel size (k_h, k_w) , stride (s_h, s_w) , padding (p_h, p_w) , and dropout probability p to the input tensor. Next, it applies the convolutional layer with ch_{in} channels, ch_{out} filters, kernel size (k_h, k_w) , stride (k_h, k_w) , and padding of zero to the output of KernelNorm. That is, both kernel size and stride values of the convolutional layer are identical to the kernel size used in KernelNorm.

From another perspective, KNConv is the same as the convolutional layer except that it normalizes the input elements specified by the kernel window before computing the convolution. Assume that U of shape (c, k_h, k_w) contains the input elements specified by the kernel window, $\hat{U}$ is the normalized version of U from KernelNorm, Z is the kernel weights of a given filter, $\star$ is the convolution (or sliding dot product) operation, and b is the bias value. KNConv computes the output as follows:

$$\text{KNConv}(U, Z, b) = \hat{U} \star Z + b = \sum_{i_c=1}^c \sum_{i_h=1}^{k_h} \sum_{i_w=1}^{k_w} \hat{U}(i_c, i_h, i_w) \cdot Z(i_c, i_h, i_w) + b \quad (4)$$

KNConv (or in fact KernelNorm) leads to extremely high number of normalization units, and consequently, remarkable computational overhead. Thus, KNConv in its simple format outlined in Equation 4 (or as a combination of the KernelNorm and convolutional layers) is computationally inefficient. Compared to the convolutional layer, the additional computational overhead of KNConv originates from (I) calculating the mean and variance of each normalization unit using Equation 2, and (II) normalizing the elements by the mean and variance using Equation 3.

Computationally-efficient KNConv reformulates Equation 4 in a way that it completely eliminates the overhead of normalizing the elements:

$$\begin{aligned} \text{KNConv}(U, Z, b) &= \hat{U} \star Z + b = \sum_{i_c=1}^c \sum_{i_h=1}^{k_h} \sum_{i_w=1}^{k_w} \left(\frac{U(i_c, i_h, i_w) - \mu_{u'}}{\sqrt{\sigma_{u'}^2 + \epsilon}} \right) \cdot Z(i_c, i_h, i_w) + b \\ &= \left(\sum_{i_c=1}^c \sum_{i_h=1}^{k_h} \sum_{i_w=1}^{k_w} U(i_c, i_h, i_w) \cdot Z(i_c, i_h, i_w) - \mu_{u'} \cdot \sum_{i_c=1}^c \sum_{i_h=1}^{k_h} \sum_{i_w=1}^{k_w} Z(i_c, i_h, i_w) \right) \cdot \frac{1}{\sqrt{\sigma_{u'}^2 + \epsilon}} + b \quad (5) \\ &= (U \star Z - \mu_{u'} \cdot \sum_{i_c=1}^c \sum_{i_h=1}^{k_h} \sum_{i_w=1}^{k_w} Z(i_c, i_h, i_w)) \cdot \frac{1}{\sqrt{\sigma_{u'}^2 + \epsilon}} + b \end{aligned}$$

According to Equation 5 and Algorithm 1, KNConv applies the convolutional layer to the original unit, computes the mean and standard deviation of the unit as well as the sum of the kernel weights, and finally adjusts the convolution output using the computed statistics. This way, it is not required to normalize the elements, improving the computation time of KNConv by orders of magnitude.

KNConvNets employ KernelNorm and computationally-efficient KNConv as the main building blocks. Figure 4 (Appendix B) illustrates the kernel normalized blocks associated with VGGNets and ResNets. The blocks are obtained by replacing traditional convolutional layers with KNConv, removing BatchNorm layers, and typically inserting KernelNorm before the max-pooling, average-pooling, and aggregation operation in residual blocks with convolutional shortcuts. KNVGGNets and KNResNets are implemented by substituting the original VGG and ResNet blocks with the kernel normalized counterparts, respectively.

Algorithm 1: Computationally-efficient KNConv layer

Input: input tensor X , number of input channels ch_{in} , number of output channels ch_{out} , kernel size (k_h, k_w) , stride (s_h, s_w) , padding (p_h, p_w) , $bias$, dropout probability p , epsilon ϵ

// 2-dimensional convolutional layer

conv_layer = Conv2d(in_channels= ch_{in} , out_channels= ch_{out} , kernel_size= (k_h, k_w) , stride= (s_h, s_w) , padding= (p_h, p_w) , bias= $false$)

// mean and variance from KernelNorm

$\mu, \sigma^2 = \text{kn_mean_var}(\text{input}=X, \text{kernel_size}=(k_h, k_w), \text{stride}=(s_h, s_w), \text{padding}=(p_h, p_w), \text{dropout_p}=p)$

// convolutional layer output

conv_out = conv_layer(input= X)

// KNConv output

kn_conv_out = (conv_out - $\mu \cdot \sum \text{conv_layer.weights}$) / $\sqrt{\sigma^2 + \epsilon}$

// apply bias

if bias **then**

 kn_conv_out += conv_layer.bias

Output: kn_conv_out

4 Evaluation

Focusing on image classification, we compare the performance of KNConvNets to BatchNorm, GroupNorm, and LayerNorm based networks in centralized, federated, and differentially private learning environments. We do not include PositionalNorm and LocalContextNorm in our comparisons because the former was not designed for image classification [19], and the latter does not outperform BatchNorm or GroupNorm for image classification [20]. In the following, we provide the details of datasets, models, training procedure, and results for different case studies.

4.1 Experimental Setup

Datasets. The **CIFAR-10/100** dataset consists of 50000 train and 10000 test samples of shape 32×32 from 10/100 classes. The **Imagenette** dataset (160-pixel version) is a subset of Imagenet [6], including 9469 train and 3925 validation images from 10 "easily classified" labels [23].

Models. We adopt the **PreactResNet-18** [25] implementation from [26], original implementation of **ResNet-18** from PyTorch [27], **ResNet-8** architecture (available at Appendix C), and **VGG-9** architecture, which is a modified version of VGG-11 with the last max-pooling and two linear layers removed. For GroupNorm based models, BatchNorm is replaced by GroupNorm with group size of 32 [12]. Regarding LayerNorm based models, GroupNorm with number of groups of 1 (equivalent to LayerNorm) is substituted for BatchNorm. The corresponding KNConvNets are implemented using the kernel normalized VGG or ResNet blocks shown in Figure 4. The dropout probability of the *final* KernelNorm layer is set to 0.5, 0.5, 0.5, and 0.25 for VGG-9, PreactResNet-18, ResNet-18, and ResNet-8, respectively, whereas it is 0.1 for the KNConv and KernelNorm layers *inside the blocks*. The activation function is ReLU for all models except ResNet-8, which employs Mish [28] as the activation function.

Case studies. We design four different case studies to make the performance comparisons:

CIFAR-100-VGG-9 (Centralized): This case study capitalizes on the *low-resolution* CIFAR-100 dataset to train the *shallow* VGG-9 model in the *centralized setting*. We adopt the data preprocessing and augmentation scheme widely used for the dataset [5, 25, 29]: Horizontally flipping and randomly cropping the samples after padding them, and normalizing the feature values by the mean and standard deviation (STD) of the dataset. The cropping and padding sizes are 32×32 and 4×4 , respectively. For the kernel normalized VGG-9, the feature values are only divided by 255 so that they lie in range $[0, 1]$. We train the models for 200 epochs using the cosine annealing scheduler [30]. The optimizer is *SGD* with momentum of 0.9; weight decay is 0.0001.

CIFAR-100-PreactResNet-18 (Centralized): This case study aims to train the *deeper* PreactResNet-18 model on the *low-resolution* CIFAR-100 dataset in the *centralized environment*. The data pre-

processing/augmentation and training procedure are the same as the CIFAR-100-VGG-9 case study, except the models are trained for 150 epochs, and weight decay is 0.0005.

CIFAR-10-ResNet-8 (Differential privacy): The goal of this case study is to train the shallow ResNet-8 model on the CIFAR-10 dataset in a *differentially private learning setting*. The feature values are divided by 255 for the kernel normalized ResNet-8, whereas they are normalized with the mean and STD of the dataset for the group and layer normalized counterparts. The models are trained for 50 epochs, where the learning rate is divided by 2 at epochs 20 and 40. The optimizer is SGD with zero-momentum.

Imagenette-ResNet-18 (Federated learning): This case study employs the *medium resolution* Imagenette dataset to train the *deeper* ResNet-18 model in a Non-IID *federated environment* consisting of 10 clients with samples only form 2 labels. The samples are normalized using the mean and STD of Imagenet for batch, group, and layer normalized models, while they are only divided by 255 for the kernel normalized ResNet-18. The samples are horizontally flipped for all models. The kernel normalized ResNet-18 is trained for 300 communication rounds with a fixed learning rate, whereas the competitors are trained for 500 rounds due to their slower convergence rate. The training algorithm is FederatedAveraging with local epochs of 1 [13], and optimizer is SGD with zero-momentum.

For all case studies, we conduct the experiments on a single GPU server equipped with *NVIDIA RTX A6000* graphic processor and *48GB* of GPU memory. For a given batch size, we perform parameter tuning by training the model using different learning rates, and determining the best learning rate based on the test/validation accuracy (Tables 3-6 at Appendix D). We repeat the experiments three times, and report the mean and STD over the runs. For centralized and federated case studies, we consider the average of the last five epochs (rounds) as the representative accuracy of the run. For the differentially private case study, the final accuracy is assumed to be the representative one.

4.2 Results

Centralized Training. Tables 1a-b list the accuracy values corresponding to the case studies in the centralized setting for different batch sizes. According to the table, (I) For batch sizes of 256 and 32, the KernelNorm based VGG-9 and PreactResNet-18 models provide slightly higher accuracy than BatchNorm counterparts; (II) For batch size of 2, kernel normalized models significantly outperform the batch normalized counterparts in terms of accuracy; (III) For all considered batch sizes, the kernel normalized VGG-9 and PreactResNet-18 also deliver considerably higher accuracy compared to the corresponding group and layer normalized models. Notice that the test accuracy values obtained for the batch normalized VGG-9 and PreactResNet-18 are highly consistent with those reported in previous studies [29, 31, 32].

Figures 3a-b illustrate the convergence rate of the models trained with batch size of 32 in the centralized environment. As shown in the figure, (I) Kernel normalized VGG-9 and PreactResNet-18 converge slightly faster than batch normalized counterparts; (II) They provide remarkably higher convergence rate compared to the GroupNorm and LayerNorm based models.

Differential Privacy. Figure 3c plots the convergence rate of the models in the differentially private setting, and Tables 1c-d list test accuracy for different ϵ values and batch sizes. According to the table and figure, Kernel normalized ResNet-8 delivers considerably higher accuracy compared to the group and layer normalized counterparts for all considered batch sizes and ϵ values. It also converges significantly faster than the competitors. Notice that BatchNorm is not applicable to the DP setting.

Federated Learning. Table 1e and Figure 3d show the accuracy values and convergence rate (communication efficiency) of the models in the federated environment, respectively. As shown in the table and figure, (I) The accuracy of the batch normalized model is considerably lower than that of batch-independent competitors, indicating BatchNorm performs poorly for federated settings; (II) The kernel normalized model remarkably outperforms the batch, group, and layer normalized counterparts for all considered batch sizes; (III) The BatchNorm based model shows unstable behavior in terms of convergence rate in federated environments; (IV) The kernel normalized ResNet-18 significantly enhances the communication efficiency compared to the other batch-independent models.

Table 1: Test accuracy for different case studies; B: batch size, $\delta=10^{-5}$.

(a) CIFAR-100-VGG-9 (Centralized)					(b) CIFAR-100-PreactResNet-18 (Centralized)				
B	BatchNorm	GroupNorm	LayerNorm	KernelNorm	B	BatchNorm	GroupNorm	LayerNorm	KernelNorm
256	70.83±0.22	66.02±0.32	66.2±0.26	71.51±0.14	256	76.29±0.32	71.78±0.42	72.05±0.42	76.47±0.16
32	70.93±0.03	67.14±0.07	67.12±0.28	71.51±0.17	32	77.36±0.16	73.45±0.28	73.05±0.07	77.74±0.06
2	68.68±0.43	66.1±0.28	66.18±0.19	71.52±0.25	2	72.2±0.17	73.7±0.09	72.72±0.49	77.14±0.26

(c) CIFAR-10-ResNet-8 (Differential privacy)					(d) CIFAR-10-ResNet-8 (Differential privacy, $\epsilon=8.0$)				
ϵ	BatchNorm	GroupNorm	LayerNorm	KernelNorm	B	BatchNorm	GroupNorm	LayerNorm	KernelNorm
8.0	NA	73.59±0.09	73.22±0.16	76.94±0.07	3072	NA	70.34±0.42	70.48±0.65	76.50±0.22
6.0	NA	71.75±0.45	71.67±0.42	75.39±0.07	2048	NA	73.59±0.09	72.68±0.36	76.94±0.07
4.0	NA	68.94±0.48	68.69±0.67	73.65±0.11	1024	NA	72.89±0.4	73.22±0.16	75.83±0.21
2.0	NA	64.44±0.2	63.33±0.81	68.05±0.23	512	NA	71.61±0.39	71.45±0.25	73.64±0.04

(e) Imagenette-ResNet-18 (Federated learning)				
B	BatchNorm	GroupNorm	LayerNorm	KernelNorm
128	25.55±2.92	72.36±0.41	72.42±0.8	75.47±0.29
32	26.39±1.91	71.86±1.08	70.82±0.41	75.68±0.21
8	45.81±4.05	72.52±0.53	71.72±0.65	76.56±0.13

5 Discussion

KNConvNets incorporate only batch-independent layers such as the proposed KernelNorm and KNConv layers into their architectures and are free of BatchNorm. They perform well not only with large batch sizes but also with very small ones in centralized training, which is not the case for BatchNorm (Table 1). Additionally, KNConvNets provide satisfactory performance in federated and differentially private environments, to which BatchNorm is unsuitable or inapplicable. KNConvNets also significantly outperform the batch-independent competitors such as LayerNorm and GroupNorm in terms of accuracy and convergence rate (Table 1 and Figure 3). Our results indicate the great potential of KNConvNets as a new class of convolutional networks to tackle the limitations of batch normalized networks in centralized, federated, and differential privacy settings.

Normalizing the feature values of the input images using the mean and standard deviation of the whole dataset is a popular data preprocessing technique, which enhances the performance of the existing batch, group, and layer normalized CNNs likely due to feeding the normalized values into the first convolutional layer. In federated settings, however, the clients require to compute the aforementioned statistics on their local data and share them with the server for aggregation, which is not a good practice from both network and privacy perspectives. This is unnecessary for KNConvNets because all KNConv layers including the first one are self-normalizing, i.e. they normalize the input first, and then, compute the convolution. This property of KNConvNets provides a valuable advantage over the batch, group, and layer normalized counterparts in federated settings.

Shift and scale learnable parameters in LayerNorm and GroupNorm (inherited from BatchNorm) can be problematic in DP settings. This is because random noise should be injected into these parameters during differentially private training, which adversely impacts the model utility. KNConvNets, on the other hand, are free of the aforementioned parameters, which is another benefit of KernelNorm compared to the existing batch-independent layers.

KernelNorm employs *randomized* mean and variance values to normalize the elements, generating a regularizing effect during training. To measure the resulting accuracy gain, we adopt the CIFAR-100-VGG-9 and CIFAR-100-PreactResNet-18 case studies with batch size of 32 and dropout probability of zero for all KNConv and KernelNorm layers. The VGG-9 and PreactResNet-18 models achieve accuracy values of 70.54% and 77.14%, respectively. Compared to 71.51% and 77.74% from Table 1, we observe that the regularizing effect of KernelNorm improves the accuracy by around 1.0% and 0.6% for the VGG-9 and PreactResNet-18 models, respectively.

The kernel size is a crucial hyper-parameter for KernelNorm and KNConv layers as the building blocks of KNConvNets. We conduct experiments to investigate the impact of kernel size using the CIFAR-100-PreactResNet-18 case study. Our results show that changing the kernel size of the KNConv layers in the shortcut connections of PreactResNet-18 from 1×1 to 2×2 improves accuracy by around 0.7%. Employing kernel size of 5×5 instead of 3×3 in the residual blocks, on the other

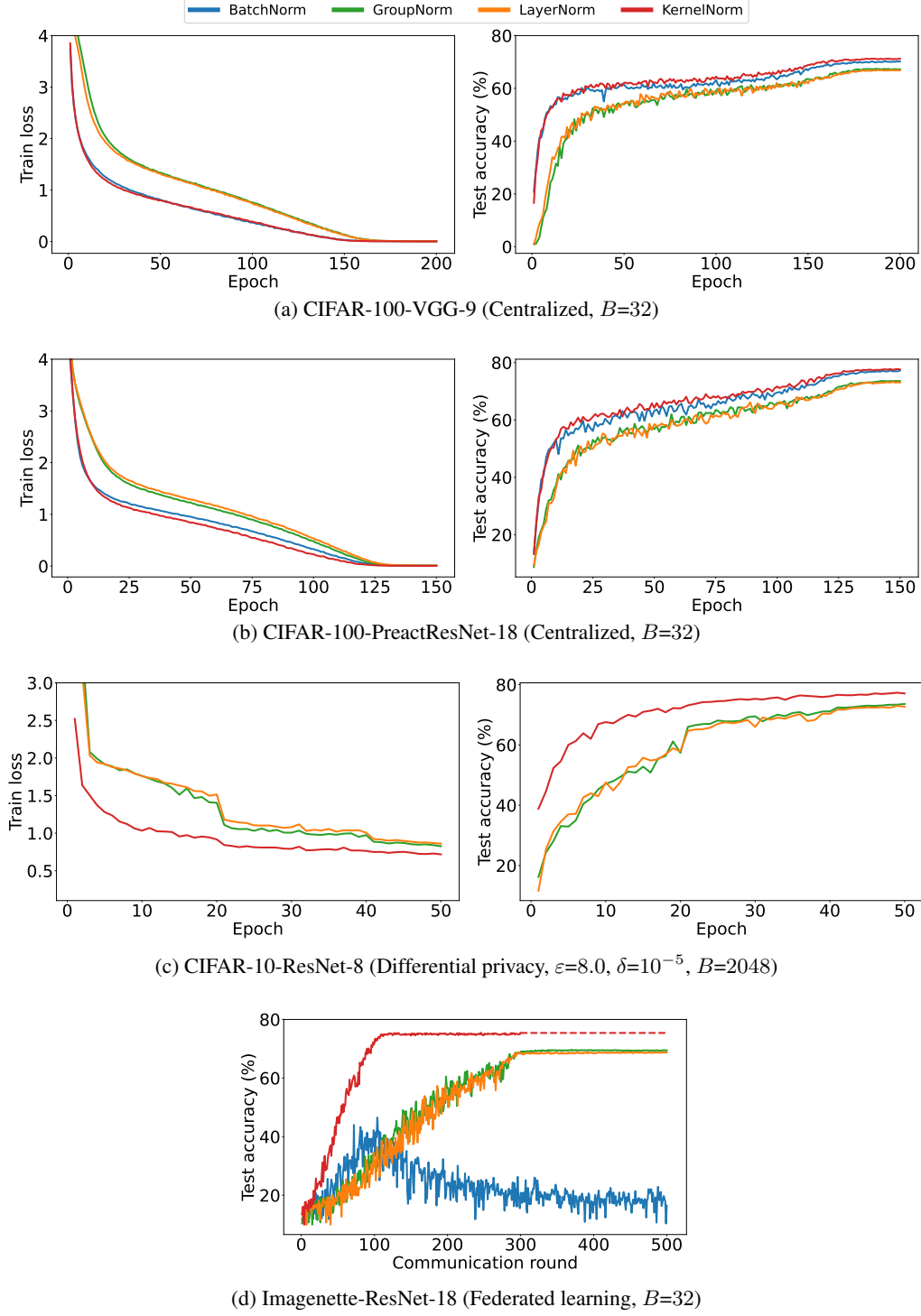


Figure 3: Convergence rate of the models for different case studies: Kernel normalized models provide slightly faster convergence rate than batch normalized models in the centralized setting. They also converge considerably faster than group and layer normalized counterparts in centralized, federated, and differentially private environments. BatchNorm shows unstable behavior in the federated setting, and is inapplicable to differentially private learning. The dashed line indicates the final test accuracy of KernelNorm at communication round 300; B : batch size.

hand, reduces the model accuracy by about 1.0%. In general, designing more efficient KNConvNets using optimal kernel sizes is an interesting topic for future studies.

The number of trainable parameters of KNConv layers is identical to the corresponding convolutional layers for given input channels, output filters, kernel size, stride, and padding values. Moreover, KernelNorm has no trainable parameters, whereas BatchNorm, GroupNorm, and LayerNorm have shift and scale parameters. Thus, KNConvNets have slightly fewer number of learnable parameters compared to the batch, group, and layer normalized counterparts.

Despite their advantages, KNConvNets are not free of drawbacks. The main limitation of KernelNorm based networks is their higher computational overhead. Our observations indicate that the training time of KNConvNets is twice or thrice as much as that of the competitors. This can be because of two factors: (1) We implemented the KernelNorm and KNConv layers using the primitives provided by PyTorch, whereas the core of the competitor layers have been implemented directly in C/C++, (2) KernelNorm is a local normalization layer, and thus, it leads to more normalization units compared to the global normalization layers such as BatchNorm (Section 2). Consequently, KNConvNets require more floating-point operations (FLOPS) to compute the statistics, resulting in longer training time.

6 Conclusion and Future Work

BatchNorm considerably enhances the model convergence rate and accuracy, but it delivers poor performance with smaller batch sizes in centralized training. Moreover, BatchNorm is unsuitable for federated learning, and inapplicable to differentially private learning due to its dependence on the batch statistics. To address these challenges, we propose two novel batch-independent layers called KernelNorm and KNConv, and employ them as the main building blocks for KNConvNets. Through extensive experimentation, we show that KNConvNets perform well for both small and large batch sizes, and deliver performance competitive to BatchNorm in the centralized setting. Furthermore, they consistently outperform the batch-independent counterparts such as LayerNorm and GroupNorm in terms of both accuracy and convergence rate in centralized, federated, and differentially private learning environments. To our knowledge, our work is the first to combine the batch-independence of LayerNorm and GroupNorm with the convergence rate and accuracy advantages of BatchNorm.

Due to limited access to computational resources, we are unable to conduct experiments on the full ImageNet dataset. A performance comparison between KNConvNets and its competitors on Imagenet is another interesting topic for future research. The performance investigation of KNConvNets for semantic segmentation or object detection is also a promising direction for future work.

Acknowledgments and Disclosure of Funding

We would like to thank Javad TorkzadehMahani for assisting with the implementations and helpful discussions on the computationally-efficient version of kernel normalized convolutional layer.

References

- [1] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.
- [2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [3] Pierre Sermanet, David Eigen, Xiang Zhang, Michael Mathieu, Rob Fergus, and Yann LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. In *2nd International Conference on Learning Representations, ICLR 2014*, 2014.
- [4] Christian Szegedy, Alexander Toshev, and Dumitru Erhan. Deep neural networks for object detection. *Advances in neural information processing systems*, 26, 2013.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [6] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [7] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.
- [8] Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. How does batch normalization help optimization? *Advances in neural information processing systems*, 31, 2018.
- [9] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.
- [10] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [11] Andy Brock, Soham De, Samuel L Smith, and Karen Simonyan. High-performance large-scale image recognition without normalization. In *International Conference on Machine Learning*, pages 1059–1071. PMLR, 2021.
- [12] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.
- [13] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
- [14] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. *arXiv preprint arXiv:1912.04977*, 2019.
- [15] Cynthia Dwork, Aaron Roth, et al. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.*, 9(3-4):211–407, 2014.
- [16] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 308–318, 2016.
- [17] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [18] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.
- [19] Boyi Li, Felix Wu, Kilian Q Weinberger, and Serge Belongie. Positional normalization. *Advances in Neural Information Processing Systems*, 32, 2019.
- [20] Anthony Ortiz, Caleb Robinson, Dan Morris, Olac Fuentes, Christopher Kiekintveld, Md Mahmudulla Hassan, and Nebojsa Jojic. Local context normalization: Revisiting local normalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11276–11285, 2020.

- [21] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.
- [22] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- [23] Jeremy Howard. Imagenette. <https://github.com/fastai/imagenette/>.
- [24] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- [25] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *European conference on computer vision*, pages 630–645. Springer, 2016.
- [26] Liu Kuang. Pytorch models for ciafr-10/100. <https://github.com/kuangliu/pytorch-cifar/>.
- [27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [28] Diganta Misra. Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681*, 2019.
- [29] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.
- [30] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.
- [31] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In *British Machine Vision Conference 2016*. British Machine Vision Association, 2016.
- [32] Pooneh Safayanikoo and Ismail Akturk. Weight update skipping: Reducing training time for artificial neural networks. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 11(4):563–574, 2021.
- [33] David J Heeger. Normalization of cell responses in cat striate cortex. *Visual neuroscience*, 9(2):181–197, 1992.
- [34] AB Bonds. Role of inhibition in the specification of orientation selectivity of cells in the cat striate cortex. *Visual neuroscience*, 2(1):41–55, 1989.
- [35] Mengye Ren, Renjie Liao, Raquel Urtasun, Fabian H Sinz, and Richard S Zemel. Normalizing the normalizers: Comparing and extending network normalization schemes. *arXiv preprint arXiv:1611.04520*, 2016.
- [36] Sergey Ioffe. Batch renormalization: Towards reducing minibatch dependence in batch-normalized models. *Advances in neural information processing systems*, 30, 2017.
- [37] Tim Salimans and Diederik P Kingma. Weight normalization: a simple reparameterization to accelerate training of deep neural networks. In *Proceedings of the 30th International Conference on Neural Information Processing Systems*, pages 901–909, 2016.
- [38] Lei Huang, Xianglong Liu, Yang Liu, Bo Lang, and Dacheng Tao. Centered weight normalization in accelerating training of deep neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2803–2811, 2017.
- [39] Siyuan Qiao, Huiyu Wang, Chenxi Liu, Wei Shen, and Alan Yuille. Micro-batch training with batch-channel normalization and weight standardization. *arXiv preprint arXiv:1903.10520*, 2019.
- [40] Andrew Brock, Soham De, and Samuel L Smith. Characterizing signal propagation to close the performance gap in unnormalized resnets. *arXiv preprint arXiv:2101.08692*, 2021.

A Related Work

Divisive normalization [33, 34] normalizes an activity of a neuron using its neighbors [35]. Local response normalization (LocalResponseNorm) [2], developed before BatchNorm, employs the divisive normalization in the context of convolutional networks, where each input element is normalized using the statistics computed over a small neighborhood of the element. Unlike KernelNorm and its competitors, LocalResponseNorm is not based on standard normalization (i.e. zero-mean and unit-variance). Batch renormalization [36] is a simple extension of BatchNorm to improve model performance on smaller batch sizes. Similar to the normalization methods reviewed in section 2 and KernelNorm, LocalResponseNorm and batch renormalization normalize the input tensor elements. Thus, we refer to them as *input normalization* techniques.

Weight normalization [37] methods, on the other hand, carries out normalization on the model weights. Weight standardization [38] in particular normalizes the weights with zero-mean and unit-norm (i.e. Euclidean norm) or using zero-mean and unit-variance (standard normalization) [39]. The KNConv layer is closely related to weight standardization but with a main difference: Given the convolution operation, which computes the dot product between the input tensor and kernel weights, KNConv performs normalization on the input tensor, whereas weight standardization normalizes the kernel weights. While KNConv layers forego the BatchNorm layers, weight standardization still requires BatchNorm to achieve satisfactory performance.

Normalizer-free ResNets [40] employ scaled weight standardization and regularization to achieve competitive performance with the batch normalized ResNets. However, they do not perform well with large batch sizes. To overcome this limitation, the authors [11] propose adaptive gradient clipping, which enables Normalizer-free ResNets to provide considerably higher accuracy than BatchNorm based networks.

B Kernel Normalized Blocks

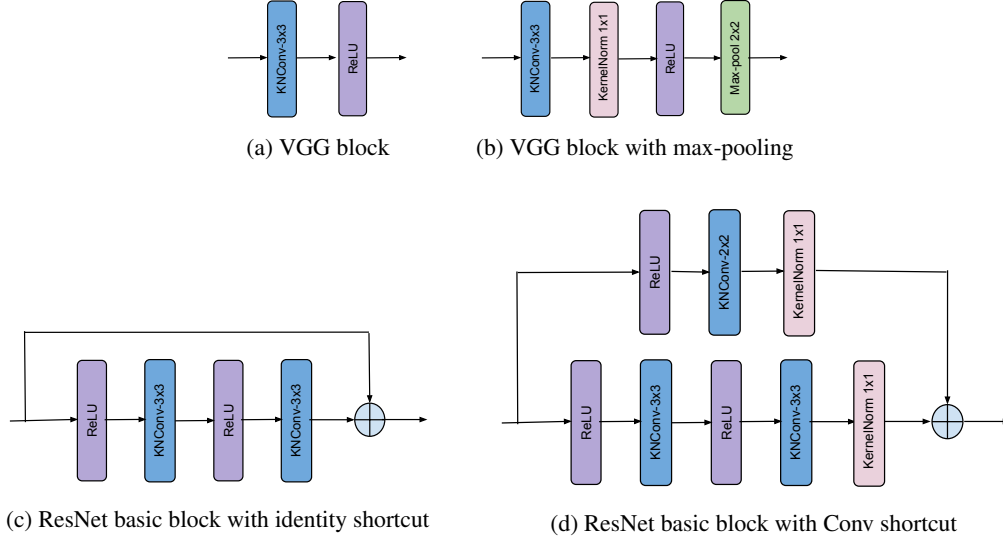


Figure 4: **Kernel normalized VGG and ResNet blocks:** All 3×3 KNConv layers have stride and padding 1×1 . The 2×2 KNConv layer has stride 2×2 and zero-padding.

C ResNet-8 Architecture

Table 2 provides specification of the layers employed in the ResNet-8 architecture. For LayerNorm and GroupNorm versions of the model, the BatchNorm layers are replaced by the corresponding layers. For the kernel normalized version, the convolutional layers are substituted with the KNConv layers, a final KernelNorm layer is inserted before the adaptive average pooling layer, and the

BatchNorm layers are removed. The output shape is (2, 2) for the adaptive average pooling layer. The $\oplus$ operation adds the output of the previous max-pooling layer to the output of Mish activation to form a residual block containing two convolutional layers.

Table 2: ResNet-8 architecture

	IN_CHANNELS/FEATURES	OUT_CHANNELS/FEATURES	KERNEL_SIZE	STRIDE	PADDING
CONV2D	3	64	(3, 3)	(1, 1)	(1, 1)
BATCHNORM2D	64	64	—	—	—
MISH	—	—	—	—	—
CONV2D	64	128	(3, 3)	(1, 1)	(1, 1)
BATCHNORM2D	128	128	—	—	—
MISH	—	—	—	—	—
MAXPOOL2D	—	—	(2, 2)	(2, 2)	(0, 0)
CONV2D	128	128	(3, 3)	(1, 1)	(1, 1)
BATCHNORM2D	128	128	—	—	—
MISH	—	—	—	—	—
CONV2D	128	128	(3, 3)	(1, 1)	(1, 1)
BATCHNORM2D	128	128	—	—	—
MISH	—	—	—	—	—
$\oplus$	—	—	—	—	—
CONV2D	128	256	(3, 3)	(1, 1)	(1, 1)
BATCHNORM2D	256	256	—	—	—
MISH	—	—	—	—	—
MAXPOOL2D	—	—	(2, 2)	(2, 2)	(0, 0)
CONV2D	256	256	(3, 3)	(1, 1)	(1, 1)
BATCHNORM2D	256	256	—	—	—
MISH	—	—	—	—	—
CONV2D	256	256	(3, 3)	(1, 1)	(1, 1)
BATCHNORM2D	256	256	—	—	—
MISH	—	—	—	—	—
$\oplus$	—	—	—	—	—
MAXPOOL2D	—	—	(2, 2)	(2, 2)	(0, 0)
ADAPTIVEAVGPOOL2D	256	256	—	—	—
LINEAR	1024	10	—	—	—

D Parameter Tuning

For all case studies, we try a set of learning rate values for a given batch size to find the learning rate achieving the best test accuracy:

- CIFAR-100-VGG-9: {1.0, 0.5, 0.25, 0.125, 0.0625, 0.03125, 0.015625, 0.0078125, 0.00390625}
- CIFAR-100-PreactResNet-18: {0.1, 0.05, 0.025, 0.0125, 0.00625, 0.003125, 0.0015625, 0.00078125, 0.000390625}
- CIFAR-10-ResNet-8: {1.0, 1.5, 2.0}
- Imagenette-ResNet-18: {0.1, 0.05, 0.025, 0.0125, 0.00625, 0.003125}

Tables 3-6 list the best learning values corresponding to the results provided in Table 1.

Table 3: The best learning rate values corresponding to the case studies in the centralized training.

(a) CIFAR-100-VGG-9					(b) CIFAR-100-PreactResNet-18				
B	BatchNorm	GroupNorm	LayerNorm	KernelNorm	B	BatchNorm	GroupNorm	LayerNorm	KernelNorm
256	1.0	0.25	0.25	1.0	256	0.1	0.05	0.05	0.1
32	0.25	0.0625	0.0625	0.125	32	0.025	0.0125	0.0125	0.05
2	0.00390625	0.00390625	0.00390625	0.0078125	2	0.00078125	0.0015625	0.0015625	0.0015625

Table 4: The best learning rate values for Imagenette-ResNet-18

B	BatchNorm	GroupNorm	LayerNorm	KernelNorm
128	0.1	0.00625	0.0125	0.00625
32	0.1	0.1	0.00625	0.003125
8	0.1	0.003125	0.003125	0.003125

Table 5: The best learning rate and clipping values for CIFAR-10-ResNet-8 with $\varepsilon=8.0$.

(a) Learning rate				(b) Clipping			
B	GroupNorm	LayerNorm	KernelNorm	B	GroupNorm	LayerNorm	KernelNorm
512	1.0	1.0	1.0	512	1.0	1.0	1.0
1024	2.0	2.0	1.5	1024	2.0	2.0	1.5
2048	2.0	2.0	2.0	2048	2.0	2.0	2.0
3072	2.0	2.0	2.0	3072	2.0	2.0	2.0

Table 6: The best learning rate, clipping, and batch size for CIFAR-10-ResNet8 with different ε .

(a) Learning rate				(b) Clipping			
ϵ	GroupNorm	LayerNorm	KernelNorm	ϵ	GroupNorm	LayerNorm	KernelNorm
2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0
4.0	2.0	1.0	2.0	4.0	2.0	1.5	2.0
6.0	1.5	2.0	2.0	6.0	2.0	2.0	2.0
8.0	2.0	2.0	2.0	8.0	2.0	2.0	2.0

(c) Batch size			
ϵ	GroupNorm	LayerNorm	KernelNorm
2.0	2048	2048	4096
4.0	1024	1024	3072
6.0	1024	2048	2048
8.0	2048	1024	3072